Retroalimentación implícita y WMF (ALS)

IIC3633

Denis Parra
Profesor Asociado
PUC Chile

Contenidos

1. Retroalimentación explícita versus implícita
2. Características de retroalimentación implícita
3. Ejemplos de métodos que usan RI
4. Introducción a ALS y su modelamiento
5. Comparación entre ALS y FunkSVD
6. ¿Cómo usar ALS?

Retroalimentación explícita versus implícita

- Llamamos retroalimentación explícita a aquellas acciones donde el usuario nos indica directamente sus preferencias, por ejemplo las calificaciones o ratings, y las acciones ”pulgar arriba”

Valerie

★★★★★ **Muy feliz**

Calificado en Estados Unidos el 15 de abril de 2023

Tamaño: L2 | **Compra verificada**

Nunca he jugado mucho al tenis, pero he tenido una pareja a la que le gusta jugar y una nueva oportunidad de jugar. Tenía unas cuantas raquetas más baratas aquí, Head and Wilson, ya que esta raqueta es una mejora definitiva para mí. Yo lo recomendaría.

Betty

★★★☆☆ **Principiante**

Calificado en Estados Unidos el 7 de marzo de 2023

Tamaño: L2 | **Compra verificada**

La raqueta tiene colores reservados, pero es para personas muy principiantes, es buena opción para tenerla en el bolso y prestársela a algún amigo que no juegue tenis diariamente

Retroalimentación explícita versus implícita

- Llamamos retroalimentación **implícita** a aquellas acciones o señales del usuario que podríamos interpretar como preferencia pero con cierto grado de incertidumbre: clicks, tiempo revisando la página de un producto, cantidad de visitas, etc.

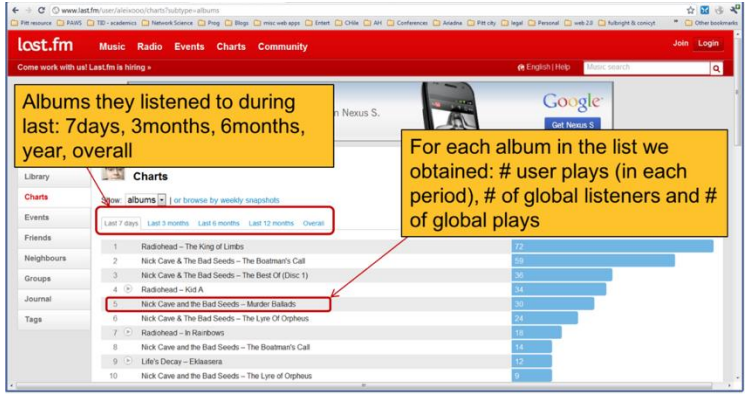
Características de la retroalimentación implícita

- Es más fácil de obtener que la retroalimentación explícita. No siempre la gente quiere dar su opinión a través de calificaciones o ratings.
- La RI no contiene esencialmente información de preferencias negativas, a diferencia de calificaciones o ratings explícitos.
- La RI contiene ruido y es difícil cuantificar preferencias de los usuarios y confianza sobre esas preferencias.
- No podemos usar métricas como RMSE o MAE para medir el rendimiento, debemos usar métricas de ranking

Relación entre RE y RI

- Algunos estudios han analizado la relación entre la retroalimentación explícita (RE, e.g.,calificaciones o ratings) e implícita (RI, e.g., clicks, cantidad de visitas, tiempo transcurrido en aplicación, etc.)

- Parra y Amatrian (2011) estudiaron la relación entre RI y RE en el contexto de música (lastfm) e identificaron una relación directa entre ambas, pero que estaba también condicionada por qué tan recientemente había sido escuchado cierta canción o artista.



- Model 1: $r_{iu} = \beta_0 + \beta_1 \cdot if_{iu}$
- Model 2: $r_{iu} = \beta_0 + \beta_1 \cdot if_{iu} + \beta_2 \cdot re_{iu}$
- Model 3: $r_{iu} = \beta_0 + \beta_1 \cdot if_{iu} + \beta_2 \cdot re_{iu} + \beta_3 \cdot gp_i$
- Model 4: $r_{iu} = \beta_0 + \beta_1 \cdot if_{iu} + \beta_2 \cdot re_{iu} + \beta_3 \cdot if_{iu} \cdot re_{iu}$

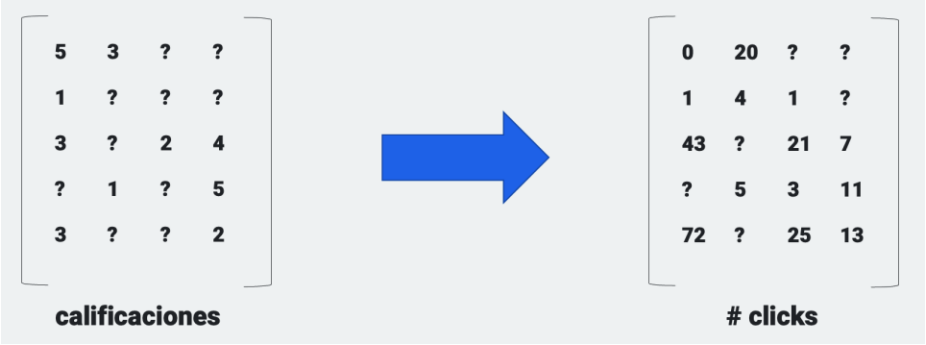
Parra, D., & Amatriain, X. (2011). Walk the talk: Analyzing the relation between implicit and explicit feedback for preference elicitation. In UMAP 2011, Girona, Spain, July 11-15, 2011

Ejemplos de métodos que usan retroalimentación implícita

- ALS
- BPR-MF
- Dwell Time

WMF (ALS)

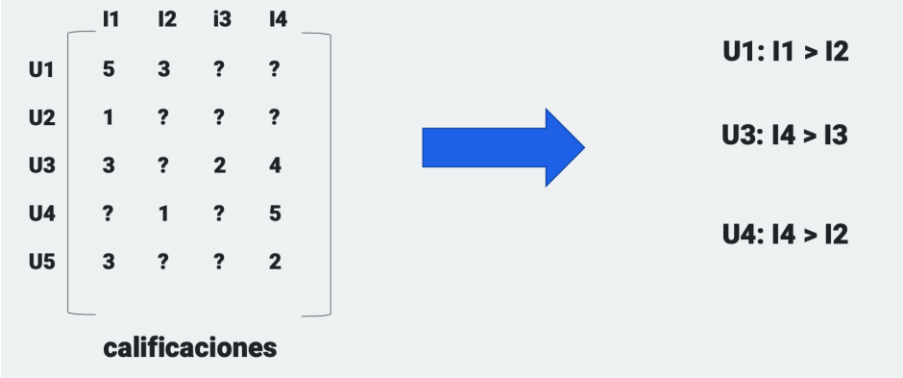
- El método de filtrado colaborativo por retroalimentación implícita inventado por Hu, Koren y Volinksy (2008) conocido como ALS o WRMF, es una versión de factorización matricial que considera cantidad de clicks o visitas en la matriz usuario-ítem en lugar de calificaciones o ratings.



Hu, Y., Koren, Y., & Volinsky, C. (2008). Collaborative filtering for implicit feedback datasets. In 2008 Eighth IEEE international conference on data mining (pp. 263-272). Ieee.

BPR-MF

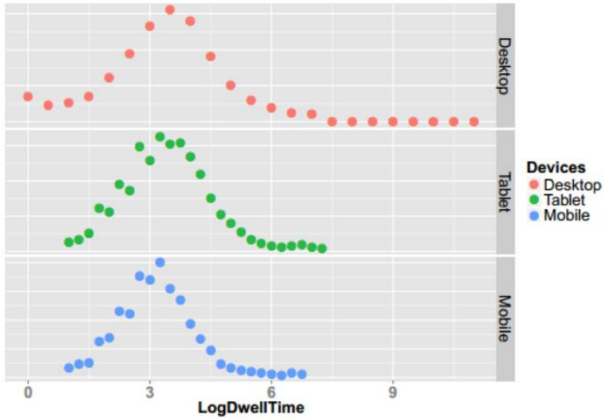
- BPR-MF significa Bayesian Personalized Ranking with Matrix Factorization.
- Es un método de factores latentes que modifica el paradigma de predecir un puntaje o calificación para cada par (usuario, ítem) y lo que intenta es ordenar, dado un usuario, si es que un ítem i es preferido sobre un ítem j.

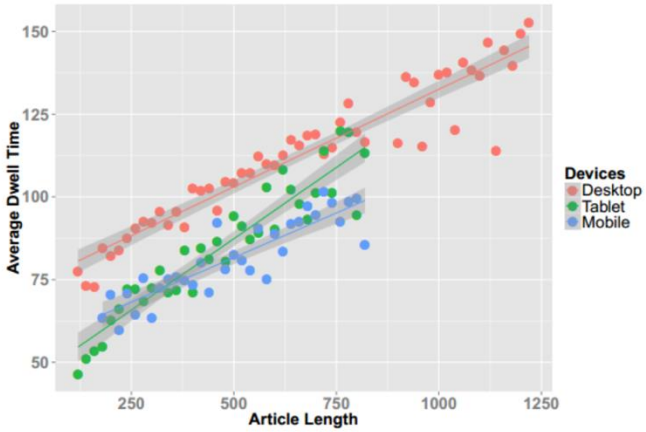


Rendle, S., Freudenthaler, C., Gantner, Z., & Schmidt-Thieme, L. (2009). BPR: Bayesian personalized ranking from implicit feedback. In Proceedings of the Twenty-Fifth Conference on Uncertainty in Artificial Intelligence (pp. 452-461).

Tiempo transcurrido (dwell time)

- Yi et al. (2014) estudiaron cómo combinar los clicks con el tiempo transcurrido en una página como señales de retroalimentación implícita para un sistema de recomendación en diferentes dispositivos.





Yi, X., Hong, L., Zhong, E., Liu, N. N., & Rajan, S. (2014). Beyond clicks: dwell time for personalization. In Proceedings of the 8th ACM Conference on Recommender systems (pp. 113-120).

Filtrado Colaborativo

- El Filtrado Colaborativo no sólo se puede implementar con calificaciones explícitas de usuarios, sino que también con retroalimentación implícita. En esta clase veremos uno de los métodos más populares: ALS.

Filtrado Colaborativo con RI

- El artículo “Collaborative Filtering for Implicit Feedback Datasets” de Hu, Koren y Volinsky introduce el método de factorización matricial para retroalimentación implícita ALS (Alternating Least Squares).

Collaborative Filtering for Implicit Feedback Datasets

Yifan Hu
AT&T Labs – Research
Florham Park, NJ 07932

Yehuda Koren*
Yahoo! Research
Haifa 31905, Israel

Chris Volinsky
AT&T Labs – Research
Florham Park, NJ 07932

Filtrado Colaborativo con R. Implícita

El artículo “Collaborative Filtering for Implicit Feedback Datasets” de Hu, Koren y Volinsky introduce el método de factorización matricial ALS (Alternating Least Squares). El modelo ALS permite:

- Aprender representaciones latentes de usuarios e ítems a partir de feedback implícito como clicks o cantidad de reproducciones de una canción o serie de TV
- Escalar la cantidad de usuarios e ítems en el modelo al paralelizar el entrenamiento con Mínimos Cuadrados Alternados (ALS en inglés)
- Evaluar el modelo propuesto en una caso real, como un proveedor de TV digital.

Modelar clicks de usuarios

Al solo contar con “cantidad de veces que se ve un producto” y no con calificaciones explícitas, Hu et al. modelaron el entrenamiento de manera distinta a FunkSVD

- Introducen una variable de preferencia p_{ui} que solo toma valores 1, 0

$$p_{ui} = \begin{cases} 1 & r_{ui} > 0 \\ 0 & r_{ui} = 0 \end{cases}$$

- donde rui indica cuantas veces el usuario *u* consumió el producto *i*

Modelar confianza sobre preferencias

Otro concepto que introduce el método de Hu et al. es el de confianza en la retroalimentación implícita. Como no es una señal explícita como las calificaciones en forma de 1 a 5 estrellas, se incorpora el concepto de confianza.

- Introducen la variable cui que mide la confianza en observar la preferencia p_{ui} y puede tomar las siguientens 2 formas:

$$c_{ui} = 1 + \alpha r_{ui}$$

$$c_{ui} = 1 + \alpha \log(1 + r_{ui}/\epsilon).$$

- donde r_{ui} indica cuántas veces el usuario *u* consumió el producto *i*, y α es un hiperparámetro que se puede optimizar posteriormente, que indica la tasa a la que crece la confianza que se tiene en la preferencia a medida que aumenta el consumo de *i*

Función de pérdida de ALS

Finalmente, la nueva función que espera encontrar los vectores latentes x* e y* que minimizan la pérdida es:

$$\min_{x_\star, y_\star} \sum_{u,i} c_{ui}(p_{ui} - x_u^T y_i)^2 + \lambda \left(\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2 \right)$$

p_{ui}: **Preferencia del usuario u por el ítem i (1 / 0)**
c_{ui}: **confianza en la preferencia del usuario**

x_u: **vector de factores latentes del usuario**
y_i : **vector de factores latentes del ítem**
λ : **coeficiente de regularización**

Comparación ALS vs FunkSVD

1. FunkSVD se desarrolló para retroalimentación explícita pero igual se podría usar para datos implícitos, mientras que ALS se usa sólo para problemas de retroalimentación implícita

2. Si bien FunkSVD se podría usar con datos de retroalimentación implícita, ALS lo supera en este ámbito al modelar preferencia y confianza en dichas preferencias

3. FunkSVD se puede evaluar tanto con métricas de error (como RMSE) como de ranking, mientras que ALS se evaluá principalmente con métricas de ranking

4. ALS es más eficiente que FunkSVD: paraleliza parte del cálculo de vectores latentes así como de aborda el cálculo de las n*m combinaciones usuario-ítem en tiempo llineal.

Comparación de funciones de pérdida

Se puede ver en la siguiente diapositiva que las funciones de pérdida son muy similares, pero la principal diferencia la hacen la noción de confianza c_{ui}, de preferencia p_{ui}, y la regularización L1,2 de ALS vs L2 de FunkSVD (notar la sumatoria adicional en la formula de ALS)

$$\sum_{r_{ui} \in R_{train}} \left(r_{ui} - \hat{r}_{ui}\right)^2 + \lambda \left(b_i^2 + b_u^2 + \boxed{||q_i||^2 + ||p_u||^2}\right)$$

$$\min_{x_\star, y_\star} \sum_{u,i} c_{ui} (p_{ui} - x_u^T y_i)^2 + \lambda \left(\boxed{\sum_u \|x_u\|^2 + \sum_i \|y_i\|^2}\right)$$

Paralelización del cálculo

- Otra importante diferencia de ALS con FunkSVD y que además es uno de los principales aportes para que ALS escale y pueda procesar millones de usuarios e ítems es paralelizar una gran parte del cálculo.
- El factor latente de un usuario u se calcula a continuación, donde la matriz Y es la matriz de preferencias usuario-ítem y la matriz Cu es la matriz de "confianzas" de los ítems consumidos por el usuario u

$$x_u = (Y^T C^u Y + \lambda I)^{-1} Y^T C^u p(u)$$

- Sin embargo, la primera expresión se puede reescribir haciendo que el primer término se calcule de forma independiente ($Y^T Y$), y el segundo tiene muy pocos elementos distintos de cero debido a Cu, porque un usuario cualquiera consume una proporción muy pequeña del total de ítems del dataset.

$$Y^T C^u Y = \boxed{Y^T Y} + Y^T (\boxed{C^u} - I) Y$$

¿Cómo usar ALS?

- **implicit** es una biblioteca de código abierto en Python que se puede usar para aprender vectores latentes con ALS y luego usarlos para recomendación, disponible en https://github.com/benfred/implicit

README.md

Implicit

Build passing documentation

Fast Python Collaborative Filtering for Implicit Datasets.

This project provides fast Python implementations of several different popular recommendation algorithms for implicit feedback datasets:

- Alternating Least Squares as described in the papers Collaborative Filtering for Implicit Feedback Datasets and Applications of the Conjugate Gradient Method for Implicit Feedback Collaborative Filtering.
- Bayesian Personalized Ranking.
- Logistic Matrix Factorization
- Item-Item Nearest Neighbour models using Cosine, TFIDF or BM25 as a distance metric.

Ejemplo de código

- **Su uso es bastante simple. En el siguiente ejemplo de código se puede ver:**

```python
import implicit
```

Importar libreria

```python
# initialize a model
model = implicit.als.AlternatingLeastSquares(factors=50)
```

Se inicializa el modelo, en este caso con 50 factores latentes

```python
# train the model on a sparse matrix of user/item/confidence weights
model.fit(user_item_data)
```

Se entrena el modelo con el método fit()

```python
# recommend items for a user
recommendations = model.recommend(userid, user_item_data[userid])
```

Dado un usuario, se recomienda con el método recommend()

```python
# find related items
related = model.similar_items(itemid)
```

Dado un ítem, se pueden obtener ítems similares con el método similar_ítems()

Fin

- denis.parra@uc.cl